Architecture SOA et Service Web

Chapitre5:

Service Web SOAP et REST

Réalisé par: Dr. Ghallabi Sameh
l'Institut d'Informatique et Mathématique à Monastir
Cible : 3émé informatique

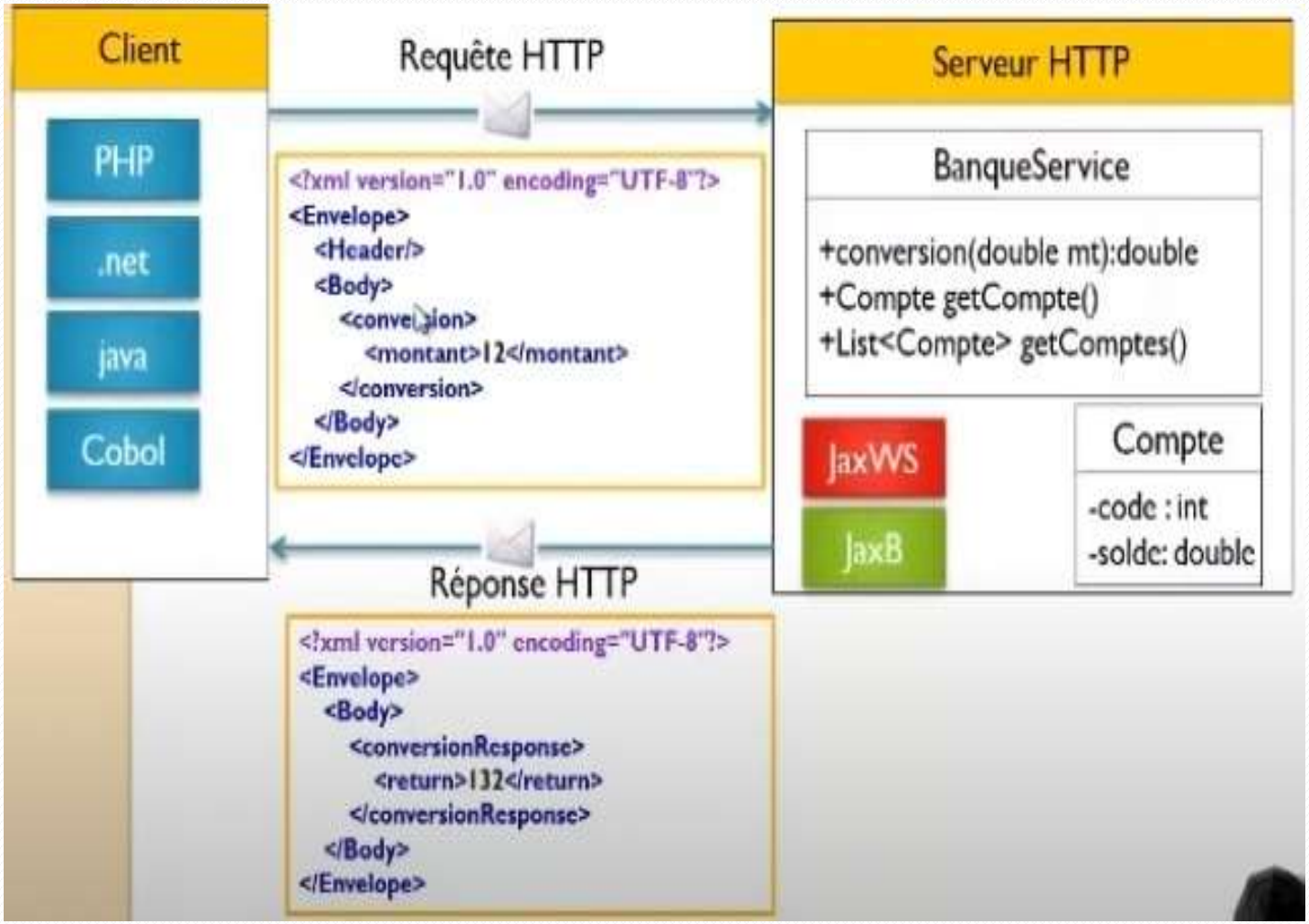
web services SOAP

✓SOAP 1.0: 1999, basé sur HTTP
✓ SOAP 1.1: 2000, plus générique, autres protocoles
✓ SOAP 1.2: recommandation W3C, 2007
✓Est un protocole de transmission de messages

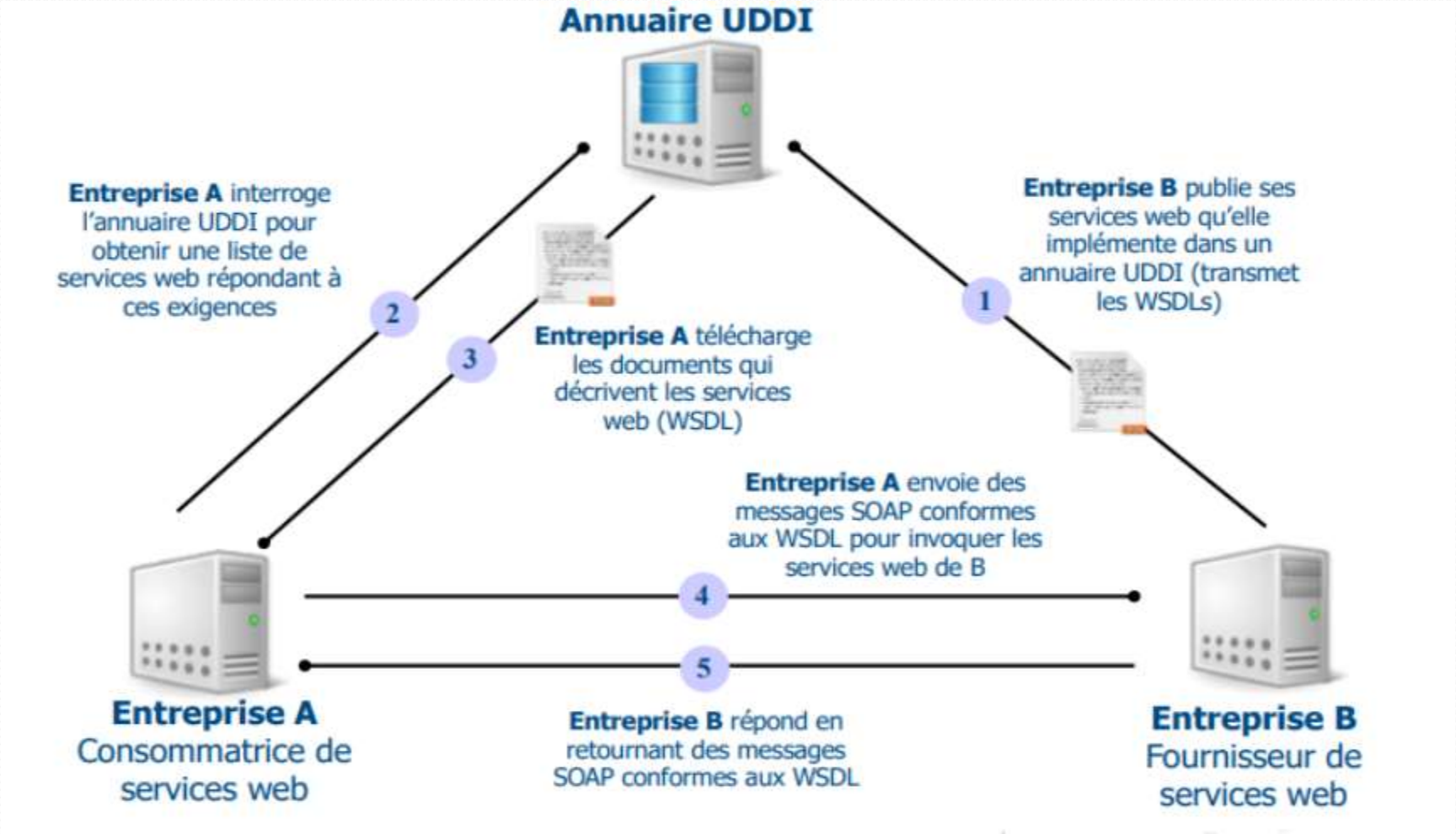
✓Permet des appels de procédures à distance (RPC) s'appuyant principalement sur le protocole HTTP et sur XML, mais aussi sur SMTP et POP.

✓Le message SOAP est destiné au fournisseur du Service après avoir contacté l'annuaire pour chercher le service correspondant au besoin du client, les informations obtenues permettent au client de connaitre la localisation du service pour pouvoir l'invoquer à l'aide de messages SOAP

web services SOAP



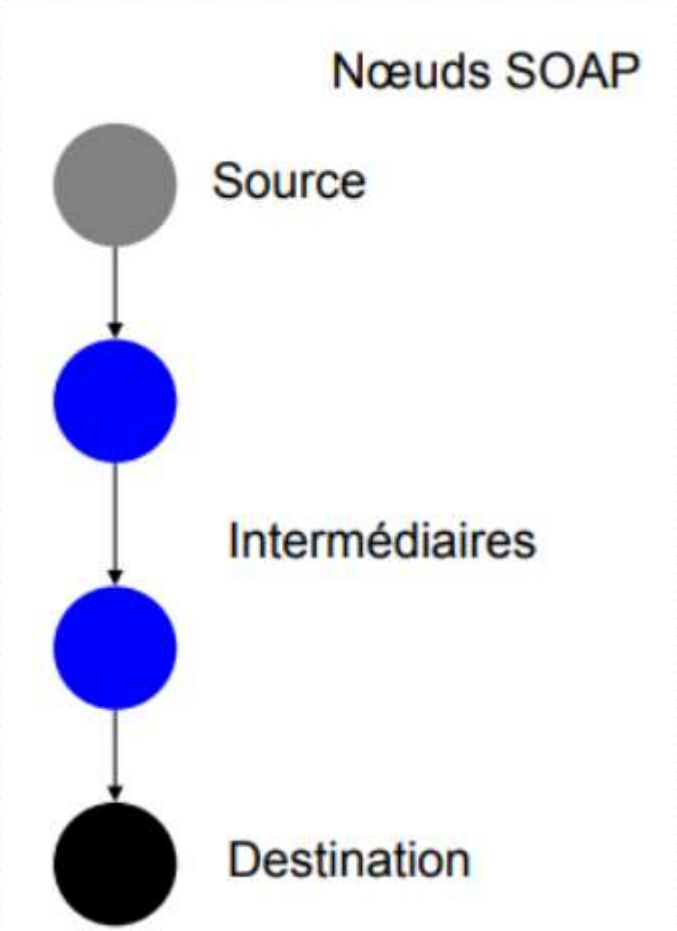
web services SOAP



Le chemin d'un message SOAP

Un message SOAP peut passer via plusieurs nœuds du nœud source au nœud destination.

- Les entités impliquées dans le transport du message sont appelés des nœuds SOAP.
- Un nœud intermédiaire SOAP peut traiter un message reçu et le transmettre au nœud suivant.
- Chaque nœud SOAP assume un certain rôle qui définit le traitement à effectuer.



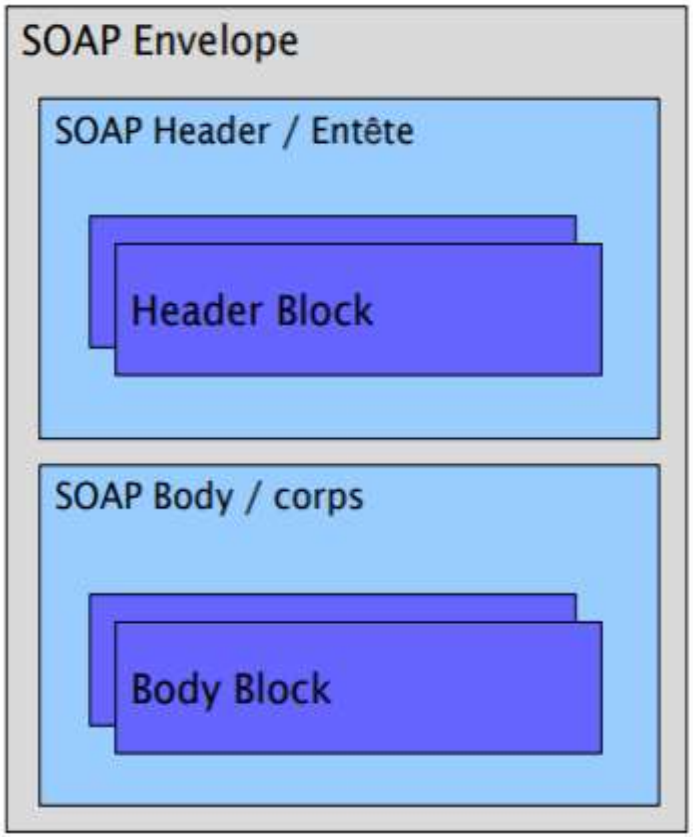
Structure des messages SOAP

Un message SOAP est un document XML contenant les éléments suivants:

- Un élément « Envelope » qui identifie le document XML en tant que message SOAP

- Un élément « Header » qui contient des informations d'en-tête.

- Un élément « Body » contenant des informations d'appel et de réponse



Squelette d'un Message SOAP

```
<?xml version="1.0"?>

<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
soap:encodingStyle="http://www.w3.org/2003/05/soar encodi ıg">

<soap:Header>
...
</soap:Header>

<soap:Body>
...
  <soap:Fault>
  ...
  </soap:Fault>
</soap:Body>

</soap:Envelope>
```

Ce qui nous intéresse , l'élément Body car il contient la méthode à invoquer, les paramètres et les données (parfois on trouve aussi les types de données des paramètres)

SOAP Exemple

SOAP REQUEST

```
<?xml version="1.0"?>

<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">

<soap:Body>
  <m:GetPrice xmlns:m="http://www.w3schools.com/prices">
    <m:Item>Apples</m:Item>
  </m:GetPrice>
</soap:Body>

</soap:Envelope>
```

SOAP Exemple

SOAP RESPONSE

```
<?xml version="1.0"?>

<soap:Envelope
xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">

<soap:Body>
  <m:GetPriceResponse xmlns:m="http://www.w3schools.com/prices">
    <m:Price>1.90</m:Price>
  </m:GetPriceResponse>
</soap:Body>

</soap:Envelope>
```

REST: Définition

✓REST est l'acronyme de REpresentational State Transfert défini dans la thèse de Roy Fielding en 2000.

✓REST est un style d'architecture inspiré de l'architecture du web.

✓L'architecture REST utilise les spécifications originelles du protocole HTTP.

✓Il n'est pas dépendant uniquement du web et peut utiliser d'autre protocoles que HTTP.

web services REST: Définition

✓REST est :

Un style d'architecture
Une approche pour construire une application

✓REST n'est pas :

Un protocole
Un format
Un standard

web services REST: Définition

✓RESTWeb Service est un Web Service qui utilise la structure REST.

✓Il a été largement utilisé, remplaçant les Web Services basés sur SOAP et WSDL.

✓REST est un ensemble de principes architecturaux.

✓SOAP (Simple Object Access Protocol) est un protocole .

web services REST: Fournisseurs











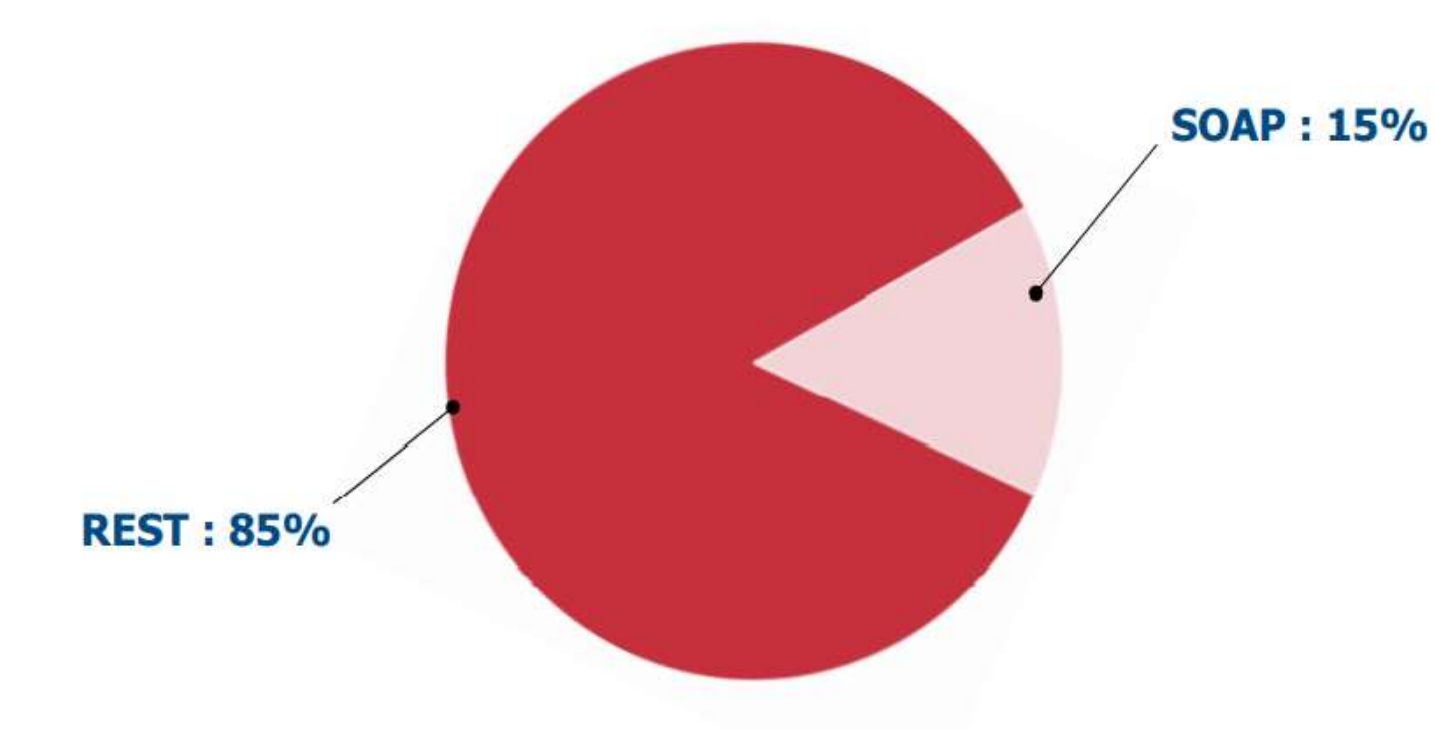






web services REST: Fournisseurs

Statistique d'utilisation des services web REST et SOAP chez AMAZON



REST: Une Architecture Simple

Basée sur 6 propriétés principales

✓Performance : Interaction simple entre les composants

✓Evolutivité : Supporte une large variété de composants

✓Simplicité : Entre les interfaces

✓Modification : Peut être modifié sans impacter les clients

✓Visibilité : Communication claire entre les composantes

✓Confiance : Reprise sur panne

REST: utilisation

✓Utiliser dans le développement des applications orientés ressources (ROA) ou orientées données (DOA).

✓ Les applications respectant l'architecture REST sont dites RESTful.

Les principes REST

Ressources (Identifiant)

- Entité identifiable dans le système (livre, agenda …)
- URI et donc possiblement URL
- Une URI identifie une seule Ressource
- Une Ressource peut avoir plusieurs URI
- Exemple :

  Emploi du temps de tigli : /edt/prof/tigli/lundi

Méthodes (Verbes)

- Quatre opérations de base « CRUD » :

  Créer (create) => POST

  Afficher (read) => GET

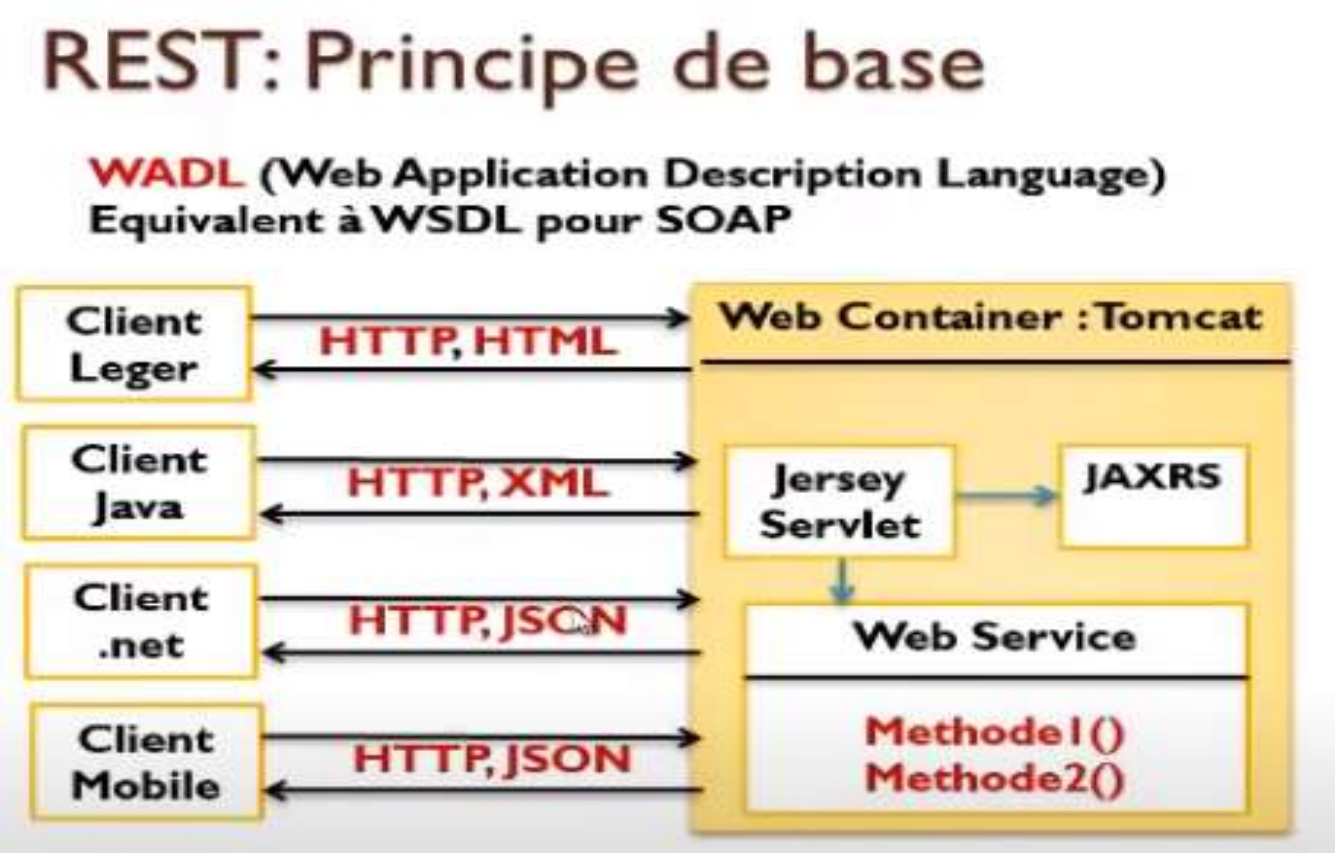
  Mettre à jour (update) => PUT

  Supprimer (delete) => DELETE
- Déjà adaptées à la manipulation de Ressources

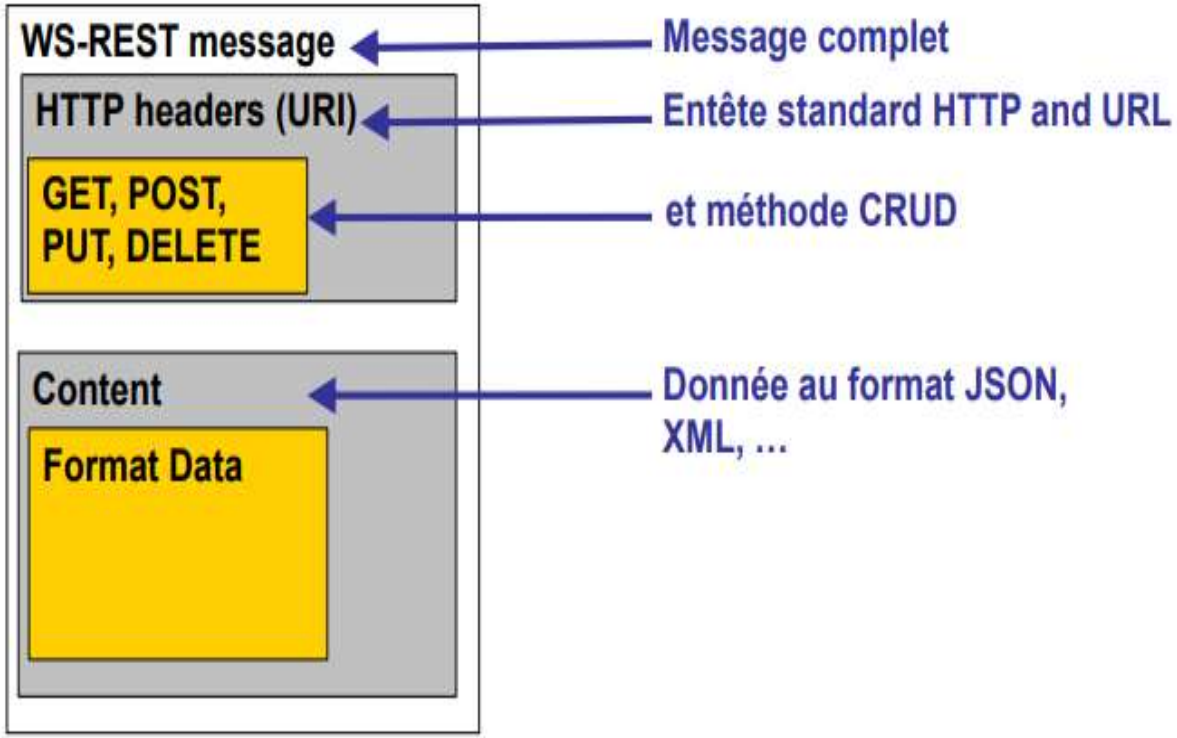
Représentation (Vue de l'état)

- Informations transférées entre client et serveur
- Exemple : contenu des ressources codé en XML, JSON, XHTML, CSV ….

Les 5 principes REST



La structure des messages WS-REST



La structure des messages WS-REST

- **Ressources** (Identifiant)
  - Identifié par une URI
  - Exemple : *http://localhost:8080/libraryrestwebservice/books*
- **Méthodes** (Verbes) pour manipuler l'identifiant
  - Méthodes HTTP : GET, POST, PUT and DELETE
- **Représentation** donne une vue sur **l'état**
  - Informations transférées entre le client et le serveur
  - Exemples : XML, JSON, ...

Le Protocole HTTP

HyperText Transfer Protocol

Protocole d'échanges d'information sur le web

Basé sur TCP/IP

URL

Unique Resource Location

Identifie les ressources de manière unique sur le Web

4 parties

- Protocole (http, ftp, mail, …)
- Host (google.com)
- Port (8080, 80)
- Path (Chemin vers la ressource sur le serveur)

Ressources

- Une ressource est un objet identifiable sur le système
  - Livre, Client, Prêt

Une ressources n'est pas forcément une entité physique, elle peut être virtuelle (Prêt, Consultation,…)

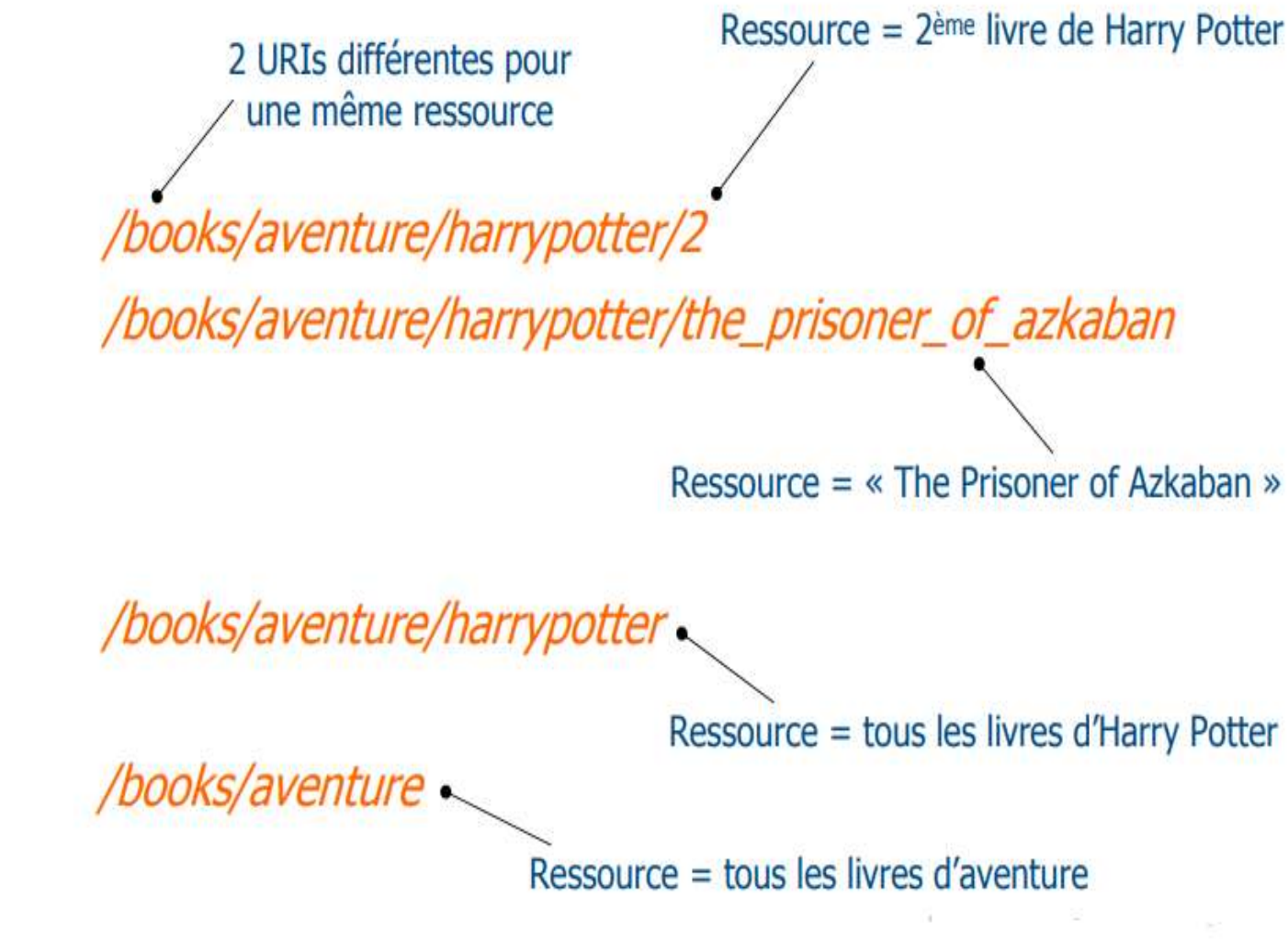
- Une ressource est identifiée par une URI : Une URI identifie uniquement une ressource sur le système

http://ntdp.miage.fr/bookstore/books/1

Clef primaire de la ressource dans la BDD

Ressources

- Exemples d'URIs

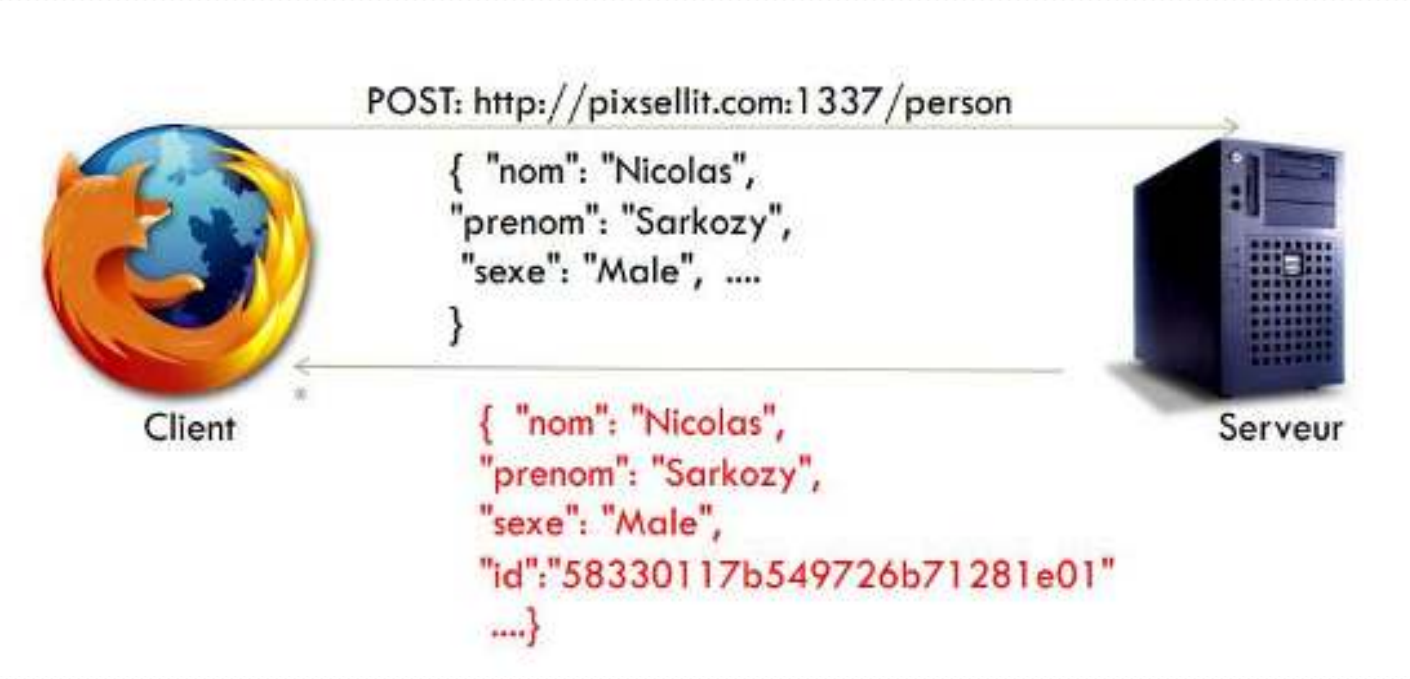


Méthode

- Une ressource quelconque peut subir quatre opérations de base désignées par **CRUD**
  - Create (Créer)
  - Retrieve (Lire)
  - Update (mettre à jour)
  - Delete (Supprimer)
- REST s'appuie sur le protocole HTTP pour exprimer les opérations via les méthodes HTTP
  - Create par la méthode **POST**
  - Retrieve par la méthode **GET**
  - Update par la méthode **PUT**
  - Delete par la méthode **DELETE**

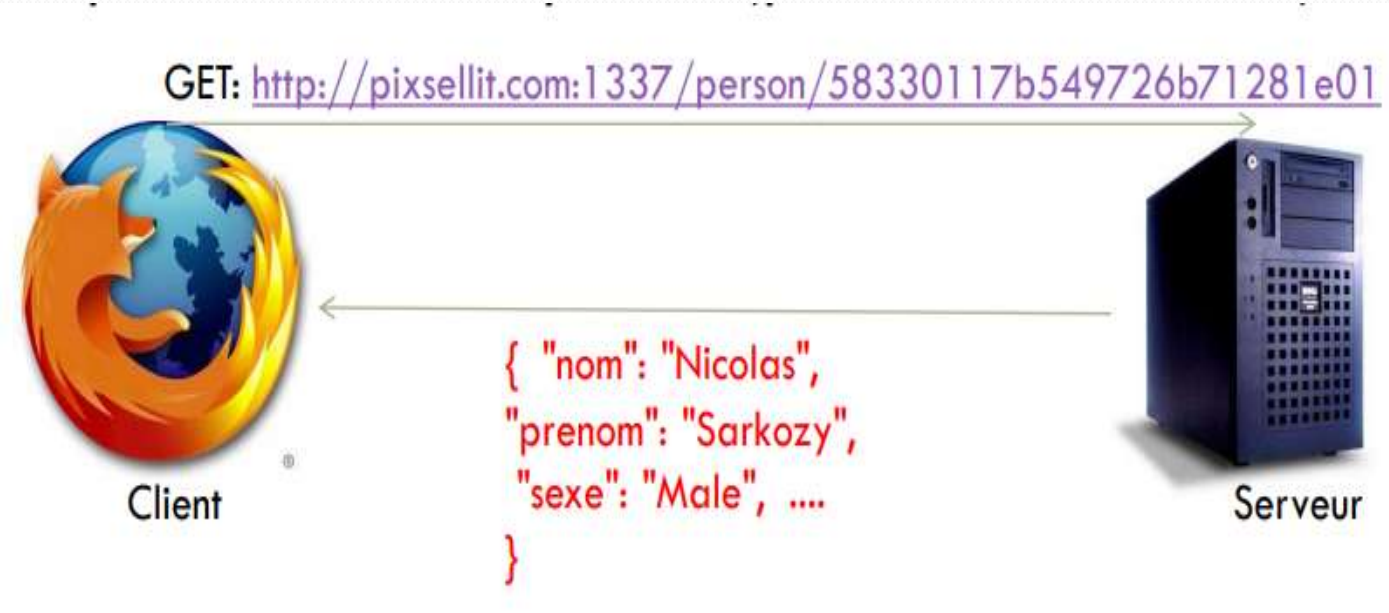
Méthode POST

Crée une nouvelle ressource sur le système



Méthode GET

Demande une représentation de la ressource tel qu'elle est sur le système (pas de modification)



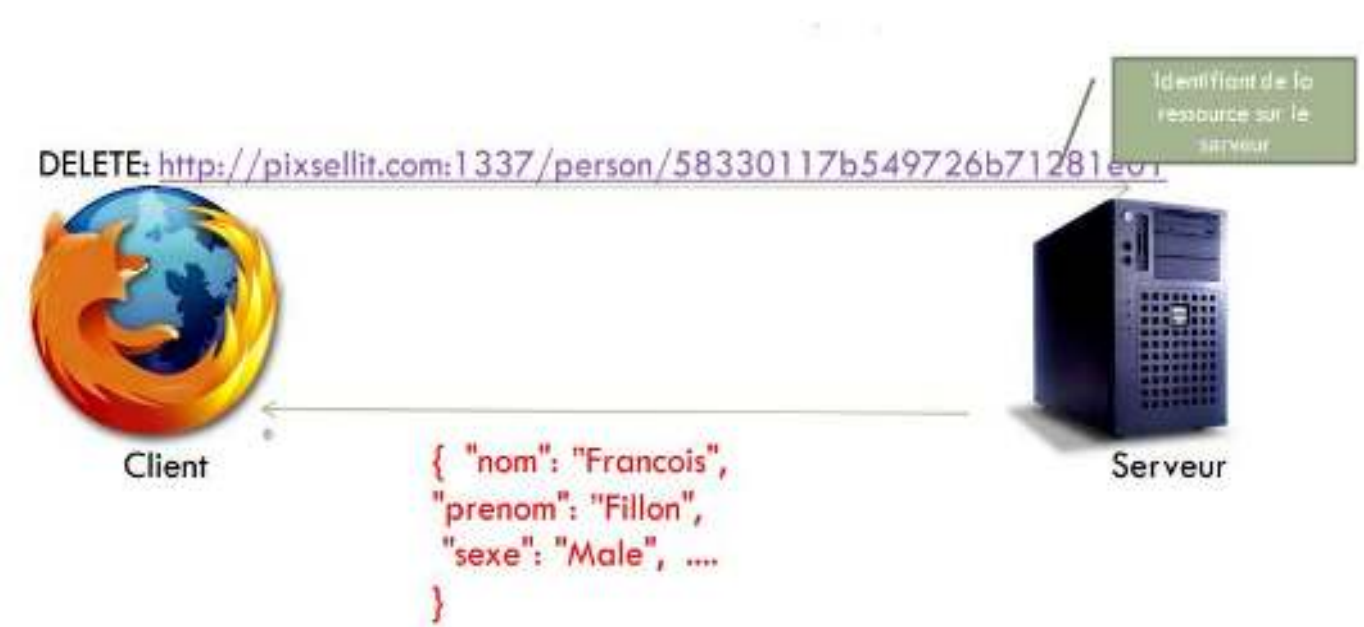
Méthode PUT

Mise à jour de la ressource sur le système



Méthode DELETE

Supprime la ressource identifiée par l'URI sur le serveur



Représentation

Une représentation désigne les données échangées entre le client et le serveur pour une ressource:

✓HTTP GET ⟶ Le serveur renvoie au client l'état de la ressource

✓PUT, POST ⟶ Le client envoie l'état d'une ressource au serveur

Peut être sous différent format :

- JSON
- XML
- XHTML
- Text
- .....

Représentation

Exemple de message HTTP RESTFul

Exemple RESTFul/XML

```
POST http://MyService/Person/
Host: MyService
Content-Type: text/xml; charset=utf-8
Content-Length: 123
<?xml version="1.0" encoding="utf-8"?>
```

HTTP Header
Commande POST

```
<Person>
 <ID>1</ID>
 <Name>M Vaqqas</Name>
 <Email>m.vaqqas@gmail.com</Email>
 <Country>India</Country>
</Person>
```

HTTP Body
XML representation
of a resource « Person »

Représentation

Exemple de message HTTP RESTFul

Exemple RESTFul/JSON

```
POST http://MyService/Person/
Host: MyService
Content-Type: text/xml; charset=utf-8
Content-Length: 123
<?xml version="1.0" encoding="utf-8"?>
```

HTTP Header
Commande POST

```
{
    "ID": "1",
    "Name": "M Vaqqas",
    "Email": "m.vaqqas@gmail.com",
    "Country": "India"
}
```

HTTP Body
JSON representation
of a resource

Service Web SOAP VERSUS RSET

```
<soapenv:Envelope
      xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
      xmlns:hel="http://helloworldwebservice.lisi.ensma.fr/">
      <soapenv:Header/>
      <soapenv:Body>
            <hel:GetOrderDetail>
                  <value>14546-xx-45</value>
            </hel:GetOrderDetail>
   </soapenv:Body>
</soapenv:Envelope>
```



Client



Serveur

SOAP

REST





Service Web SOAP VERSUS RSET

- Les Services Web étendus (SOAP) et les S différent par le fait que
  - Services Web étendus reposent sur des standa
  - REST est un style d'architecture
- Services Web étendus (SOAP)
  - **Avantages**
    - Standardisé
    - Interopérabilité
    - Sécurité (WS-Security)
    - Outillé
  - **Inconvénients**
    - Performances (enveloppe SOAP supplémen
    - Complexité, lourdeur
    - Cible l'appel de service

Service Web SOAP VERSUS RSET

- Services Web **REST**
  - **Avantages**
    - Simplicité de mise en œuvre
    - Lisibilité par l'humain
    - Evolutivité
    - Repose sur les principes du Web
    - Représentations multiples
  - **Inconvénients**
    - Sécurité restreinte par l'emploi des méthodes HTTP
    - Cible uniquement l'appel de ressource

WADL

WADL (Web Application Description Language) est un langage de description XML de services

✓Soumis en 2009 au W3C

✓Permet de décrire les éléments des services

Resource, Méthode, Paramètre, Réponse

✓ Est un format de fichier basé sur XML qui permet de décrire des applications REST.

✓Permet d'interagir de manière dynamique avec les applications REST.

✓De plus, WADL est encore très mal supporté par l'ensemble des frameworks existants ce qui limite son utilisation.

WADL

Exemple : Afficher le WADL de services REST

```
<application>
<doc jersey:generatedBy="Jersey: 1.4 09/11/2010 10:30 PM"/>
<resources base="http://localhost:8088/librarycontentrestwebservice/">
      <resource path="/contentbooks">
            <resource path="uribuilder2">
                  <method name="POST" id="createURIBooks">
                        <request>
                              <representation mediaType="application/xml"/>
                        </request>
                        <response>
                              <representation mediaType="*/*"/>
                        </response>
                  </method>
            </resource>
            <resource path="uribuilder1">
                  <method name="POST" id="createBooksFromURI">
                        <request>
                              <representation mediaType="application/xml"/>
                        </request>
                        <response>
                              <representation mediaType="*/*"/>
                        </response>
                  </method>
            </resource>
            ...
      </resource>
</resources>
</application>
```

http://localhost:8088/librarycontentrestwebservice/application.wadl